
Fundamental of Machine learning

REGEX, XML AND JSON

A solid orange horizontal bar spanning the width of the slide at the bottom.

Regex

Regex

A RegEx, or Regular Expression, is a sequence of characters that forms a search pattern.

RegEx can be used to check if a string contains the specified search pattern.

RegEx in Python

- `import re`

Regex

RegEx Functions

- ***re.findall*** Returns a list containing all matches
- ***re.search*** Returns a Match object if there is a match anywhere in the string
- ***re.split*** Returns a list where the string has been split at each match
- ***re.sub*** Replaces one or many matches with a string

Regex

Metacharacters

Character	Description	Example
[]	A set of characters	"[a-m]"
\	Signals a special sequence (can also be used to escape special characters)	"\d"
.	Any character (except newline character)	"he..o"
^	Starts with	"^hello"
\$	Ends with	"world\$"
*	Zero or more occurrences	"aix*"
+	One or more occurrences	"aix+"
{}	Exactly the specified number of occurrences	"a1{2}"
	Either or	"falls stays"
()	Capture and group	

Regex

Special Sequences

- **\d** Matches any decimal digit; this is equivalent to the class `[0-9]`.
- **\D** Matches any non-digit character; this is equivalent to the class `[^0-9]`.
- **\s** Matches any whitespace character; this is equivalent to the class `[ \t\n\r\f\v]`.
- **\S** Matches any non-whitespace character; this is equivalent to the class `[^ \t\n\r\f\v]`.
- **\w** Matches any alphanumeric character; this is equivalent to the class `[a-zA-Z0-9_]`.
- **\W** Matches any non-alphanumeric character; this is equivalent to the class `[^a-zA-Z0-9_]`

Regex

Re.search()

- *Return a* `_sre.SRE_Match` **Object**

```
import re

sentence = 'This is a sample string'
x = re.search("is", sentence)

print("Start position:", x.start())
print("End position:", x.end())
```

Regex

Re.findall()

- Return a list containing every occurrence of pattern

```
import re

sentence = 'This is a sample string, sammmple'
x = re.findall("am", sentence)
print(x)
```


Regex

Re.split()

- Returns a list where the string has been split at each match

```
import re

sentence = 'This is a sample string, sammmple'
x = re.split("\s", sentence)
print(x)
```

Regex

Re.sub()

- Returns a list where the string has been split at each match

```
import re

sentence = 'This is a sample string, sammmple'
x = re.sub("\s", "-", sentence)
print(x)
```

XML

XML

XML stands for eXtensible Markup Language.

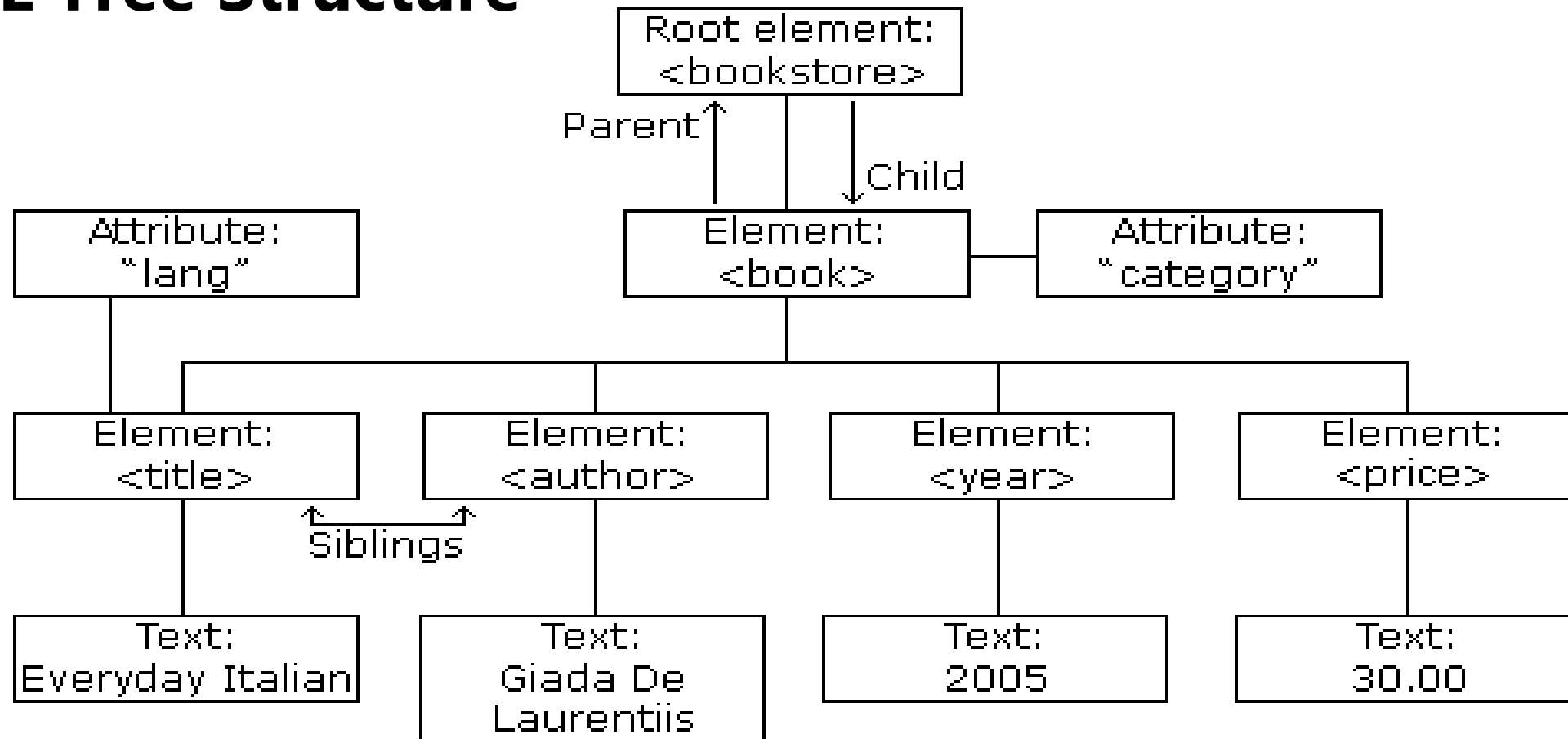
XML was designed to store and transport data.

XML was designed to be both human- and machine-readable.

XML is a markup language much like HTML

XML

XML Tree Structure



XML

The two standard approaches for XML processing are supported in the standard library:

- `xml.dom.*` – a standard DOM API
- `xml.sax.*` – a standard SAX API

But there's `xml.etree.ElementTree` (ET for short)

- a lightweight Pythonic API
- supports both DOM-like (but ET faster than DOM) and SAX-like processing (i.e. event-based, streaming i.e. all-in-memory)
- fast C implementation used by default whenever possible in Python3 (no need for `xml.etree.cElementTree` as in Python 2)

XML

- **loading from a file:**

```
import xml.etree.ElementTree as ET
tree = ET.ElementTree(file='sample.xml')
```

- **from a string::**

```
root = ET.fromstring('<my><own><xmlcontent/></own></my>')
```

XML

Retrieve:

```
root = tree.getroot()

for child in root:
    print(child.tag, child.attrib, child.text)

for descendant in root.iter():
    ....
```


XML

Accessing attribute values:

```
root = ET.fromstring('<koren id="x15" name="John"/>')
for attr in root.attrib:
    print(attr+"="+root.attrib[attr])
```

XML

Searching:

- **Simple searching**

```
for elem in tree.iter(tag='surname'):  
    ....
```

- **Complex searching using XPath**

```
for elem in tree.iterfind('*/section/figure[@id="f15"]'):  
    ....
```

XML

Creating+storing :

```
root = ET.Element('root')
newelem = ET.SubElement(root, 'data')
ET.dump(root)
```

JSON

JSON

JavaScript Object Notation

A simple text-oriented format for data exchange between a browser and a server

Inspired by JavaScript object literal syntax, but nowadays used well beyond the JavaScript world

Became one of the most popular data exchange formats in the last years

JSON vs XML

```
<?xml version="1.0" encoding="UTF-8"?>
<book id="123">
  <title>Object Thinking</title>
  <author>David West</author>
  <published>
    <by>Microsoft Press</by>
    <year>2004</year>
  </published>
</book>
```

```
{
  "id": 123,
  "title": "Object Thinking",
  "author": "David West",
  "published": {
    "by": "Microsoft Press",
    "year": 2004
  }
}
```

JSON

Data – hierarchical structures

Curly braces hold objects

- • name and value separated by colon
- • name-value pairs separated by comma

Square brackets hold arrays

- values separated by comma

Whitespaces (space, tab, LF, CR) around syntactic elements ignored

BOM not allowed

No syntax for comments

JSON – data types

number

string

boolean

array

object

null

JSON in Python

`json` – JSON API is available in the standard library

- **Implicit type conversions**
 - ✓ A JSON object goes to Python dict
 - ✓ a JSON array goes to Python list
 - ✓ a JSON string goes to Python unicode
 - ✓ a JSON number goes to Python int or long
 - ✓ a JSON true goes to Python True
 - ✓ etc.
 - ✓ and vice versa

JSON – serializing/deserializing

```
import json
```

```
named_entity = {"form": "Bob", "type": "firstname", span: [0, 1, 2]}
```

```
serialized = json.dumps(named_entity)
```

```
restored = json.loads(serialized)
```

JSON – serialization options

There's some space for customizing the serialization (within the limits given by the JSON spec):

- encoding – the character encoding (utf-8 by default)
- indent – pretty-printing with the specified indent level for object members
- sort_keys – output of dictionaries sorted lexicographically by key
- separator – tuple (item_sep, key_sep)

XML vs. JSON – similarities

Both XML and JSON are frequently used for data interchange

Both formats are human readable (if designed properly)

Both are currently supported by many programming languages

XML vs. JSON – differences

As usual, we face the trade-off of simplicity against expressiveness

With some over-simplification: JSON is a lightweight cousin of XML

JSON is slightly less verbose and simpler (and faster) to parse, but currently there's more functionality associated with the XML standard:

Namespaces, referencing, validations schemes, stylesheet transformations, query languages etc.

So there's no clear superiority of one against the other

Your final choice should depend on what you really need (and, of course, on the system context)

THANK YOU!
